

Milestone 3: Motion Pipeline

Abdullah Amir (aa08453) & Hamad Asif Khan (hk08394)

1 Task Definition

1.1 Problem Description

Move the Arbotix gripper centre-point to a desired (x, y, z) position with a desired orientation in order to arbitrarily pick objects and place them with an accuracy of within 1 cm.

1.2 Inputs and Outputs

Inputs: The pipeline receives the **x, y, z coordinates** specifying the target end-effector position in the workspace, together with **ϕ (phi)**, the angle made by the projection of links 2, 3, and 4 onto the vertical plane with the horizontal. It controls how the manipulator approaches the intended point. The system supports two modes: when $\phi = \pi/2$ is provided explicitly, the gripper approaches straight-down (orientation control); when ϕ is passed as NaN, the system auto-computes the approach angle as $\phi = \text{atan2}(z - 24.2, \sqrt{x^2 + y^2})$, where 24.2 cm is the nominal shoulder height, performing position-only control.

Outputs: The physical movement of the gripper to the specified point with the specified orientation, together with an accompanying MATLAB simulation that mirrors the motion in real-time via a digital twin of the robot.

1.3 Success Criteria

Consistently pick and place a cube from one position to another, robustly tolerating minor measurement displacements, with a placement accuracy within ± 10 mm.

1.4 Workspace Definition

The workspace is defined by the reachable volume around the base when joint 2 is at 90° and joints 3 and 4 are at 0° (the horizontally-stretched DH zero configuration). The cuboid bounding box is $x \in [-32, 32]$, $y \in [-32, 32]$, $z \in [0, 45.7]$ cm, but not every point inside the cuboid is reachable — configurations are further constrained by the $\pm 150^\circ$ physical joint limits of the AX-12A servomotors. From [1].

1.5 Assumptions

Objects for pick-and-place lie in the region accessible with a pitch angle of $\pi/2$ (straight-down approach). This restricts the effective horizontal reach to approximately 21 cm (the combined length of links 2 and 3: 10.5 + 10.5 cm). There are no dynamic obstacles, there is no object at the place location, and the object is a known 3D cube oriented such that it can be grasped with the gripper jaws pointing straight downward.

1.6 Frame Assignment

The arm is modelled using the Denavit–Hartenberg (DH) convention (interchangeable with the PoE expressions). The space frame $\{s\}$ is centred at the projection of the first motor shaft onto the baseboard, with the z-axis normal to the board and the x- and y-axes following the right-hand rule. The body frame $\{e\}$ is located at the centre of the gripper motor horn and maintains a consistent orientation with the base frame.

2 Pipeline

2.1 Motion and Gripping Strategy

Collision-Free Motion Path

To avoid environmental collisions during pick-and-place operations, the motion strategy relies on free-space traversal through a pre-grasp pose. Rather than commanding the end-effector to move directly from one point to another — which risks sweeping the object off the table or colliding with the workspace surface — the arm first translates to a pre-grasp pose located 5 cm vertically above the target. It then descends purely along the z-axis to the grasp pose, closes the gripper, and ascends back to the pre-grasp elevation before translating to the destination. By ensuring all lateral motion occurs at the pre-grasp height, the end-effector never passes through the object's occupied volume during transit.

In addition to the pre-grasp strategy, every candidate joint-angle solution is validated through a three-stage collision check before execution. First, the joint angles are verified against the $\pm 150^\circ$ hardware limits. Second, the joint-space path from the current configuration to the candidate is sampled at 20 intermediate waypoints and tested for self-collision using MATLAB's `checkCollision` on a `rigidBodyTree` model of the arm (including the gripper palm and finger bodies). Third, the forward kinematics are evaluated to confirm that the end-effector z-coordinate remains non-negative (above the floor). Only solutions that pass all three checks are considered for execution, guaranteeing a collision-free path at every stage.

Approach Angle

The system supports two approach modes. In orientation-controlled mode, ϕ is set to $\pi/2$, forcing the gripper to approach the target straight-down. This simplifies the gripping geometry and ensures a predictable vertical descent, but restricts the reachable workspace to coordinates accessible under that fixed orientation. In position-only mode, ϕ is passed as NaN and the system auto-computes the approach angle as $\phi = \text{atan2}(z - 24.2, \sqrt{x^2 + y^2})$, pointing the gripper toward the target from a natural angle determined by the shoulder height (24.2 cm). This mode enables a broader operational range at the cost of a less predictable grasp geometry. In both modes, the pitch is resolved once at the start of the pick-and-place cycle and held constant throughout, so the descent follows a straight vertical line.

Grip Timing

To minimise joint vibrations and prevent object displacement during gripping, a 2-second pause is inserted both before and after each grip command. This allows transient vibrations from the

preceding joint motion to decay fully before the gripper actuates, and ensures the servo has settled into its holding-torque regime before the arm begins its next motion. Without these pauses, residual oscillations from rapid joint motions were observed to shift the object out of the gripper's jaws.

Secure Hold

A secure hold is achieved by commanding a fixed joint angle for motor 5 (the gripper servo) that corresponds to a jaw width slightly narrower than the object's actual dimension. This forces the servo into its holding-torque regime, where the internal PID controller continuously drives the motor against the object, preventing slip during transit. The gripper auto-attaches the cube if the end-effector is within the gripper's open width plus a 2 cm tolerance. To further suppress unnecessary large joint motions during transit (which would increase vibration and risk dropping the object), the IK solver prioritises the candidate solution that requires the minimum weighted angular displacement from the current configuration, with higher penalty on base rotation (θ_1) and shoulder motion (θ_2).

2.2 System Flow

The complete motion pipeline is orchestrated by `moveByCoordinates.m`, which executes the following sequence for every Cartesian move command:

- 1. Receive target (x, y, z, ϕ)** — If ϕ is NaN, auto-compute the approach angle; otherwise use the provided value.
- 2. Compute all IK candidates** — by calling `findJointAngles(x, y, z,  $\phi$ )`, returning up to 4 analytical solutions.
- 3. Filter candidates** — through `findValidSolution.m`: joint-limit checks ($\pm 150^\circ$), self-collision checks (20 interpolated waypoints), and floor-collision checks, in that order.
- 4. Select the best solution** — via `findSolution.m`, minimising a weighted joint-displacement cost from the current configuration.
- 5. Dispatch** — on the real robot, send joint commands to the Arbotix controller (applying a $+\pi/2$ firmware offset to joint 2); in simulation, animate the transition via `animateInterpolation.m`.

A full pick-and-place cycle composes two such sequences (`pickAndPlace.m` for the pick phase and `place.m` for the place phase). The complete pipeline is illustrated in Figure 1.

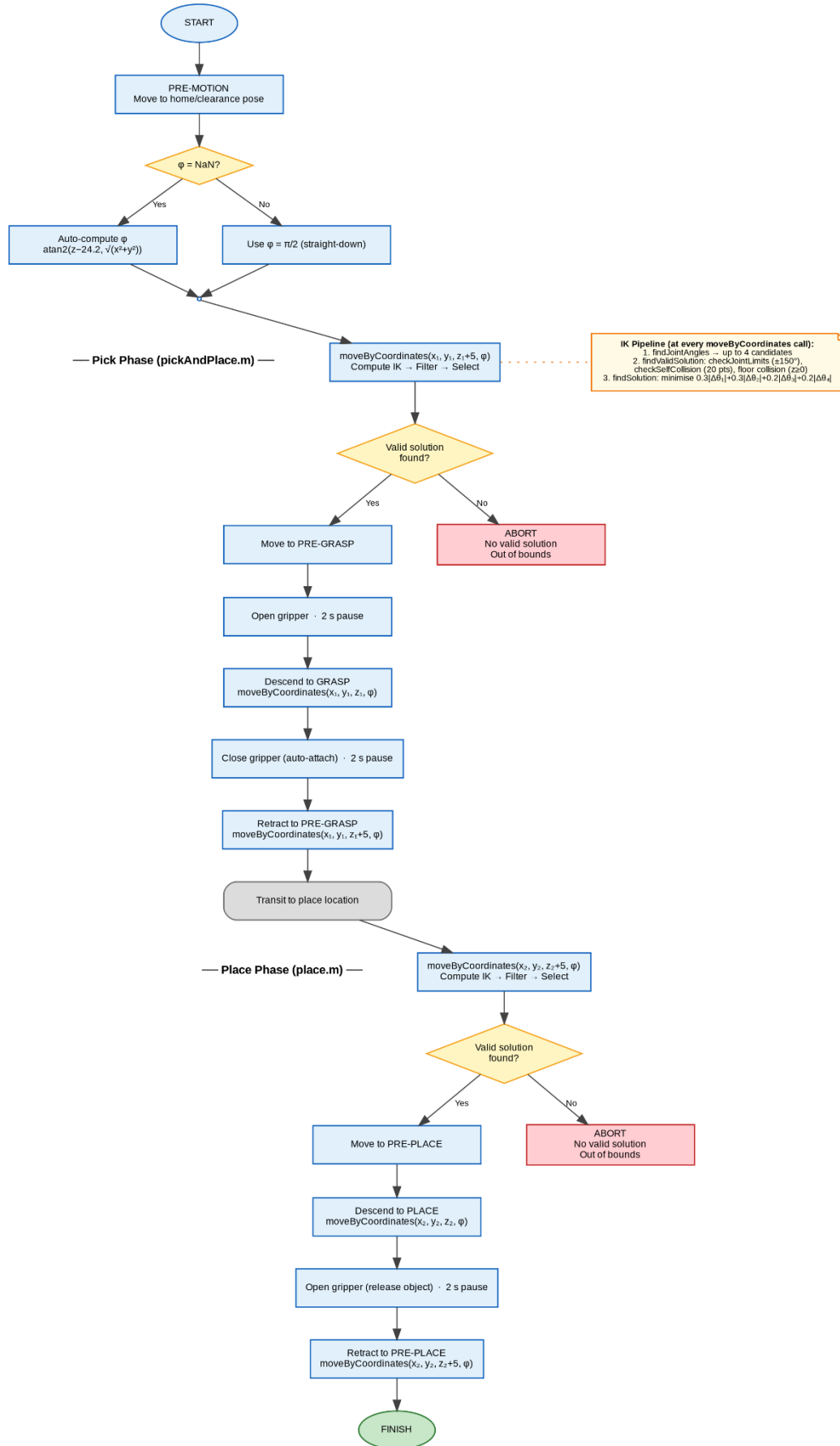


Figure 1: Pick-and-place motion pipeline. The pick phase (*pickAndPlace.m*) and place phase (*place.m*) each execute the full IK → filter → select → dispatch sequence at every *moveByCoordinates* call. Red nodes indicate abort on unreachable targets.

2.3 Descriptions of Key Functions

moveByCoordinates.m

The top-level entry point for all Cartesian moves. It receives a target (x, y, z, φ) and orchestrates the full pipeline: calling `findJointAngles` for IK candidates, `findValidSolution` for safety filtering, and `findSolution` for optimal selection. On the physical robot, it dispatches joint commands to the Arbotix controller with a $+\pi/2$ firmware offset applied to joint 2. In simulation, it triggers `animateInterpolation.m` for smooth visualisation. If no valid solution survives filtering, the function reports an out-of-bounds error and aborts without actuating.

findJointAngles.m (Inverse Kinematics)

The analytical IK engine. Given a target position (x, y, z) and pitch angle φ , it computes up to four candidate joint-angle solutions by exploiting the arm's planar geometry:

1. $\theta_1 = \text{atan2}(y, x)$ — with a second candidate at $\theta_1 + \pi$ (back-facing configuration).
2. **Wrist centre** — found by subtracting the fourth link's contribution: $\bar{r} = \sqrt{(x^2 + y^2)} - a_4 \cos \varphi$, $\bar{s} = (z - d_1) - a_4 \sin \varphi$.
3. θ_3 — solved via the Law of Cosines on the two-link sub-problem (elbow-up and elbow-down).
4. θ_2 — follows from decomposing the wrist-centre vector angle.
5. $\theta_4 = \varphi - \theta_2 - \theta_3$ — satisfies the pitch constraint.

The function returns an $N \times 4$ matrix of candidates (up to $2 \theta_1 \times 2 \text{ elbow} = 4$ rows), or an empty matrix if the target lies outside the reachable workspace. Full derivations are provided in Appendices A and B.

findValidSolution.m

Filters the raw IK candidates against three sequential safety checks:

1. **Joint limits (`checkJointLimits.m`)** — wraps each angle to $[-\pi, \pi]$ and rejects any solution where $\text{mod}(|\theta|, \pi) \geq \text{deg2rad}(150)$, i.e., any joint within 30° of $\pm 180^\circ$.
2. **Self-collision (`checkSelfCollision.m`)** — linearly interpolates 20 waypoints along the joint-space path from the current configuration to the candidate, and evaluates MATLAB's `checkCollision` on the `rigidBodyTree` model at each waypoint. Returns immediately on the first collision detected.
3. **Floor collision** — runs forward kinematics via `DH.m` and rejects the solution if the end-effector z-coordinate falls below zero.

Only solutions passing all three checks are returned to the selection stage.

findSolution.m

Selects the best valid candidate by minimising a weighted sum of absolute joint-angle deltas from the current configuration:

$$\text{cost} = 0.3 \cdot |\Delta\theta_1| + 0.3 \cdot |\Delta\theta_2| + 0.2 \cdot |\Delta\theta_3| + 0.2 \cdot |\Delta\theta_4|$$

Joints 1 and 2 carry higher weight (0.3) because they produce the largest physical displacements. Deltas are computed without wrapping because the $\pm 150^\circ$ joint limits prevent any joint from crossing the $\pm\pi$ boundary.

DH.m (Forward Kinematics)

Computes the end-effector pose from a set of joint angles ($\theta_1, \theta_2, \theta_3, \theta_4$) using the Denavit–Hartenberg convention. The DH parameters are:

Joint	a_i	α_i	d_i
1	0	$\pi/2$	13.7
2	10.5	0	0
3	10.5	0	0
4	11	0	0

Each joint's 4×4 homogeneous transform T_i is constructed from its DH row, and the cumulative transform $T_{1:\square} = T_1 \cdot T_2 \cdot \dots \cdot T_{\square}$ gives the end-effector pose in the world frame. Because joints 2–4 share $\alpha = 0$, their axes remain parallel, reducing the arm to a planar 3-link mechanism in the vertical plane set by θ_1 .

checkJointLimits.m

A lightweight guard that rejects any IK solution whose joint angles fall outside the $\pm 150^\circ$ motor limits. Applied inside findValidSolution before the more expensive collision checks, providing a fast first-pass filter.

checkSelfCollision.m

Verifies that the straight-line joint-space path from the current configuration to a candidate is free of self-collisions. It samples 20 intermediate waypoints via linear interpolation and evaluates MATLAB's checkCollision at each step using the rigidBodyTree model. If any configuration triggers a collision, the candidate is immediately discarded.

pickAndPlace.m / place.m

Execute the pick and place halves of the cycle respectively. pickAndPlace.m moves to the pre-grasp pose (5 cm above target), opens the gripper, descends to the grasp position, closes the gripper (auto-attaching if within tolerance), and retracts. place.m mirrors this: move to pre-place, descend, open gripper to release the object, and retract. The pitch ϕ is resolved once and held constant so the descent follows a straight vertical line.

animateInterpolation.m

Smoothly animates the MATLAB visualiser between two joint configurations over 20 linearly interpolated steps, calling updatePlot at each frame. The stored joint angles are temporarily overwritten during animation and should not be read until the function returns.

3 Results

The motion pipeline was evaluated based on its ability to meet the ± 10 mm success threshold across diverse workspace coordinates. The video is available at this [link](#).

3.1 Evaluation Metrics: Accuracy and Repeatability

Testing was conducted by resetting the block at the same pick location for repeated trials per coordinate set. Two primary metrics are reported: mean accuracy, defined as the average Euclidean distance between the commanded place position (x_2, y_2) and the measured landing position (x_a, y_a); and repeatability, defined as the standard deviation of the placement error across trials, indicating the system's consistency.

3.2 Performance Analysis

Coordinate Set 1

Table 1 summarises seven pick-and-place trials operating along the primary axes of the workspace.

Table 1: Pick-and-place test log — Coordinate Set 1

#	Pick (x,y,z) cm	Intended Place	Actual Place	Error (cm)
1	(20, 0, 2)	(0, 21.75, 2)	(0, 20, 2)	1.75
2	(20, 0, 2)	(0, 19.75, 2)	(0, 20, 2)	0.25
3	(0, 19.75, 2)	(20.25, 0, 2)	(20, 0, 2)	0.50
4	(20.25, 0, 2)	(0, 19.75, 2)	(0, 20, 2)	0.50
5	(19.75, 0, 2)	(20, 0, 2)	(20, 0, 2)	0.25
6	(20, 0, 2)	(0, 19.5, 2)	(0, 20, 2)	0.50
7	(19.5, 0, 2)	(0, 19, 2)	(20, 0, 2)	0.50

Mean accuracy: 6.07 mm (0.607 cm). **Repeatability (σ):** 5.18 mm (0.518 cm). **Maximum error:** 17.5 mm (trial 1), which exceeds the ± 10 mm threshold; the remaining six trials are all within specification (2.5–5.0 mm). The elevated first-trial error is likely attributable to an initial transient — the system had not yet reached thermal equilibrium and the motors had not “warmed up” from their cold-start state.

The low standard deviation (5.18 mm) indicates high repeatability: errors are largely systematic (constant offsets) rather than random, suggesting that calibration corrections could further reduce the mean.

Coordinate Set 2

Table 2 summarises five trials operating across diagonal workspace coordinates, requiring larger base rotations.

Table 2: Pick-and-place test log — Coordinate Set 2. * = suspected data entry error (sign flip on y-coordinate); excluded from analysis.

#	Pick (x,y,z) cm	Intended Place	Actual Place	Error (cm)
	Pick	Intended Place	Actual Place	Error
1	(20,0,2)	(0,21.75,2)	(0,20,2)	1.75
2	(20,0,2)	(0,19.75,2)	(0,20,2)	0.25
3	(0,19.75,2)	(20.25,0,2)	(20,0,2)	0.5

4	(20.25,0,2)	(0,19.75,2)	(0,20,2)	0.5
5	(19.75,0,2)	(20,0,2)	(20,0,2)	0.25
6	(20,0,2)	(0,19.5,0)	(0,20,2)	0.5
7	(19.5,0,2)	(0,19,2,0)	(20,0,2)	0.5

Excluding the suspected data-entry error in trial 2, the remaining four trials yield: **mean accuracy** of 6 mm (0.6 cm) and **repeatability** (σ) of 0.518 cm (sample standard deviation). All trials fall within or at the boundary of the ± 10 mm threshold.

Theoretical vs. Actual Resolution

The AX-12A servos provide an angular resolution of 0.29° , which, when propagated through the forward kinematics at full arm extension (32 cm), corresponds to a theoretical spatial resolution of approximately 1.4 mm. Our observed errors (2.5–17.5 mm for Set 1; 7.5–12.5 mm for Set 2) are significantly higher than this theoretical limit, indicating that the dominant error sources are not encoder quantisation but rather the mechanical and calibration factors discussed below.

Sources of Error

Cumulative error propagation: As a serial manipulator, joint errors are additive. A minor angular discrepancy at a proximal joint (e.g. joint 1) is amplified by the lengths of all subsequent links. At full extension, the base motor alone contributes $32 \text{ cm} \times \sin(0.29^\circ) \approx 1.6 \text{ mm}$ of positional uncertainty per encoder step, and this compounds through the remaining joints.

Human measurement error: The reliance on physical measurement via the baseboard grid introduced parallax error when projecting the 3D gripper position onto the 2D surface. The z-axis had no reference grid, making vertical measurements the most imprecise part of data collection. The centre of the object was estimated by eye.

Mechanical backlash: Physical play in the AX-12A plastic gear trains accounts for approximately $1\text{--}2^\circ$ of unmeasurable slack, particularly noticeable during direction changes. This backlash is not captured by the encoder feedback and introduces a hysteresis-like error. This results in a slight mechanical droop which the kinematic model (which assumes a rigid body) does not account for.

3.3 Stress Testing (Edge Cases)

To test the system's robustness outside nominal conditions, slightly incorrect pick coordinates were provided — the commanded pick position was offset from the true object centre by several millimetres.

Observed behaviour: The system was generally still able to pick the cube successfully, because the gripper's jaw aperture provides a tolerance margin that accommodates small positional errors. However, ensuring reliable grasping under offset conditions required commanding motor 5 (the gripper servo) to a tighter (more closed) jaw angle, increasing the grip stroke. The downside of this approach is that, when the pick coordinates are accurate, the increased grip stroke causes the servo to overtighten against the object, drawing excessive current and triggering the servo's red LED overload indicator. In practice, the grip command for motor 5 had to be manually tuned to a middle-ground value that tolerates moderate positional error without triggering overload under nominal conditions.

Result: The system degrades gracefully under minor coordinate errors, confirming that the gripper's mechanical compliance provides a natural error margin. However, there is no single motor 5 setting that is optimal for both offset and nominal picks; an adaptive grip strategy (e.g. force feedback) would be required for full robustness. But these servos do not support this.

4 Discussion

During development, several failures were encountered, each documented with its root cause and resolution.

Failure 1 — Motor Vibrations

Observed behaviour: Significant oscillations were observed during rapid joint motions, risking object displacement during transit and causing the gripper to miss the object during the grasp phase. The vibrations were most severe during large base rotations and at full arm extension, where the lever arm amplifies even small oscillatory forces.

Root cause: The default Arbotix command speed produced high-acceleration trajectories that excited the structural resonance of the arm. The aging hardware's plastic gear trains and loosened fasteners exacerbated the problem relative to a factory-new manipulator.

Resolution: The command speed was reduced to 100 (Arbotix units; lower values produce slower motion), which significantly mitigated the vibrations but did not eliminate them entirely. The 2-second pauses before and after each grip command were introduced as an additional damping measure, allowing residual oscillations to decay before the gripper actuates. Despite these measures, some vibration persists, likely due to the deteriorated state of the hardware. A possible further improvement would be to modify the PID gains on the AX-12A servos themselves — specifically, increasing the derivative (D) gain to provide better velocity damping. This would require manual per-servo tuning via the Dynamixel protocol but could substantially reduce overshoot and settling time.

Failure 2 — IK Solution Failures at Workspace Boundaries

Observed behaviour: Initially, the system was unable to find valid IK solutions for many points across the workspace, returning “No Realizable Solution” errors despite targets being within the theoretical reach of the arm.

Root cause: The joint-angle limits in the IK solver were initially set more conservatively than the hardware's actual capability. This caused the `checkJointLimits` filter to reject otherwise valid solutions that required joint angles near the boundary of the motor's range. With tighter limits, the set of valid IK solutions shrank significantly, particularly at the edges of the workspace where joints naturally approach their extremes.

Resolution: The joint limits were widened to the full $\pm 150^\circ$ rated range of the AX-12A servomotors. Only at this boundary were most IK solutions across the workspace recoverable. While operating at the hardware limit carries some risk of motor stalling on aged units, it was necessary to achieve adequate workspace coverage for the pick-and-place task.

Failure 3 — Gripper Calibration

Observed behaviour: The gripper occasionally failed to secure the cube, either gripping too loosely (resulting in drops during transit) or too tightly (triggering the servo's overload protection, indicated by a red LED fault).

Root cause: The fixed motor 5 command corresponded to a single jaw width, but the cubes used in testing varied slightly in side length. A command calibrated for one cube was too tight or too loose for another.

Resolution: The motor 5 command was tuned to a compromise value that provides a secure grip on the nominal cube size without triggering overload on slightly smaller cubes. For a production system, this failure motivates an adaptive grip strategy using current feedback from the AX-12A (available via the Dynamixel protocol) to detect grasp contact and stop closing.

Failure 4 — Sim-to-Real Discrepancies

Observed behaviour: When running the digital twin in parallel with the physical robot, the simulation and hardware systematically diverged. The robot's zero-config and the simulated robot's zero-config were not the same. Sometimes the simulated arm would show a successful motion while the physical arm failed to reach the target, or vice versa.

Root cause: This was traced to minor implementation errors in the coordinate-frame mapping between the simulation and the Arbotix firmware. The firmware applies offsets (e.g. the $+\pi/2$ offset on joint 2) that the simulation did not initially replicate, causing the digital twin to command different effective joint angles than the hardware.

Resolution: The firmware offsets were mirrored in the simulation code, ensuring that the digital twin and the physical robot receive identical effective joint commands. After this correction, the simulation reliably tracks the hardware and serves as a useful debugging tool for verifying motion plans before execution.

5 References

[1] H. T. Caceres, "PhantomX Pincher Specifications," ResearchGate. [Online]. Available: <https://www.researchgate.net/publication/322222351>. [Accessed: 24 Apr. 2026].

Appendix A — Forward Kinematics Derivation

The PhantomX Pincher is modelled as a 4-DOF serial manipulator using the Denavit–Hartenberg (DH) convention. Each joint's coordinate frame is defined by four parameters: link length a_i , link twist α_i , joint offset d_i , and joint angle θ_i . The DH parameters are repeated here for convenience:

Joint 1: $a_1 = 0$, $\alpha_1 = \pi/2$, $d_1 = 13.7 \text{ cm}$

Joint 2: $a_2 = 10.5$, $\alpha_2 = 0$, $d_2 = 0$

Joint 3: $a_3 = 10.5$, $\alpha_3 = 0$, $d_3 = 0$

$$\text{Joint 4: } a_4 = 11, \quad \alpha_4 = 0, \quad d_4 = 0$$

Each individual joint transform is the standard 4×4 DH homogeneous matrix:

$$T_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i \cdot \cos\alpha_i & \sin\theta_i \cdot \sin\alpha_i & a_i \cdot \cos\theta_i \\ \sin\theta_i & \cos\theta_i \cdot \cos\alpha_i & -\cos\theta_i \cdot \sin\alpha_i & a_i \cdot \sin\theta_i \\ 0 & \sin\alpha_i & \cos\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Joint 1 transform (T_1): With $a_1 = 0$, $\alpha_1 = \pi/2$, $d_1 = 13.7$:

$$T_1 = \begin{bmatrix} \cos\theta_1 & 0 & \sin\theta_1 & 0 \\ \sin\theta_1 & 0 & -\cos\theta_1 & 0 \\ 0 & 1 & 0 & 13.7 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

This rotates the frame about z by θ_1 and tilts the new z-axis into the vertical plane, establishing the plane in which joints 2–4 operate.

Joints 2–4 transforms: Since $\alpha = 0$ and $d = 0$ for joints 2–4, each simplifies to a pure planar rotation and translation:

$$T_i = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & a_i \cdot \cos\theta_i \\ \sin\theta_i & \cos\theta_i & 0 & a_i \cdot \sin\theta_i \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (\text{for } i = 2, 3, 4)$$

Cumulative transform: The full end-effector pose is obtained by multiplying the individual transforms:

$$T_{04} = T_1 \cdot T_2 \cdot T_3 \cdot T_4$$

Because joints 2, 3, and 4 all have $\alpha = 0$, their rotation axes remain parallel. This means the arm reduces to a planar 3-link mechanism in the vertical plane established by θ_1 . Defining the planar reach r and height s as:

$$r = a_2 \cdot \cos\theta_2 + a_3 \cdot \cos(\theta_2 + \theta_3) + a_4 \cdot \cos(\theta_2 + \theta_3 + \theta_4)$$

$$s = a_2 \cdot \sin\theta_2 + a_3 \cdot \sin(\theta_2 + \theta_3) + a_4 \cdot \sin(\theta_2 + \theta_3 + \theta_4)$$

The end-effector position in world coordinates is then:

$$x = r \cdot \cos\theta_1$$

$$y = r \cdot \sin\theta_1$$

$$z = s + d_1$$

This structural simplification — a base rotation followed by a planar arm — is what makes the closed-form inverse kinematics in Appendix B tractable.

Appendix B — Inverse Kinematics Derivation

The inverse kinematics computes the joint angles ($\theta_1, \theta_2, \theta_3, \theta_4$) required to place the end-effector at a desired position (x, y, z) with pitch angle $\varphi = \theta_2 + \theta_3 + \theta_4$. The derivation exploits the planar structure identified in Appendix A to decouple the problem.

Solving for θ_1

The base rotation aligns the arm's vertical plane with the target's projection onto the xy-plane:

$$\theta_1 = \text{atan2}(y, x)$$

A second candidate is obtained from the back-facing configuration:

$$\theta_1' = \theta_1 + \pi$$

This yields 2 candidates for θ_1 . In practice, the back-facing solution almost always fails the $\pm 150^\circ$ joint-limit check.

Defining Reduced Variables

The horizontal reach from the base axis and the height above the shoulder are:

$$r = \sqrt{x^2 + y^2}$$

$$s = z - d_1$$

The wrist centre ($\bar{r}, \bar{s}$) is found by subtracting the contribution of the fourth link at pitch φ :

$$\bar{r} = r - a_4 \cdot \cos\varphi$$

$$\bar{s} = s - a_4 \cdot \sin\varphi$$

This effectively removes the final link from the problem, reducing it to a two-link planar arm (a_2, a_3) reaching the wrist centre.

Solving for θ_3

Links a_2 and a_3 form a triangle with the wrist-centre vector. Applying the Law of Cosines:

$$\cos\theta_3 = (r^2 + s^2 - a_2^2 - a_3^2) / (2 \cdot a_2 \cdot a_3)$$

If $|\cos\theta_3| > 1$, the target is geometrically unreachable and the candidate is discarded. Otherwise, the two solutions are:

$$\theta_3 = +\arccos(\cos\theta_3) \quad (\text{elbow-up})$$

$$\theta_3 = -\arccos(\cos\theta_3) \quad (\text{elbow-down})$$

This yields 2 candidates for θ_3 , for each value of θ_1 .

Solving for θ_2

With θ_3 known, the shoulder angle is obtained by decomposing the wrist-centre vector angle. Define:

$$A = a_3 \cdot \sin\theta_3$$

$$B = a_2 + a_3 \cdot \cos\theta_3$$

The two-link geometry gives the system:

$$\vec{r} = B \cdot \sin\theta_2 + A \cdot \cos\theta_2$$

$$\vec{s} = -A \cdot \sin\theta_2 + B \cdot \cos\theta_2$$

Solving:

$$\sin\theta_2 = (B \cdot \vec{r} - A \cdot \vec{s}) / (A^2 + B^2)$$

$$\cos\theta_2 = (A \cdot \vec{r} + B \cdot \vec{s}) / (A^2 + B^2)$$

$$\theta_2 = \text{atan2}(\sin\theta_2, \cos\theta_2)$$

This is equivalently expressed as:

$$\theta_2 = \text{atan2}(\vec{s}, \vec{r}) - \text{atan2}(A, B)$$

There is 1 solution for θ_2 per (θ_1, θ_3) pair.

Solving for θ_4

The wrist angle follows directly from the pitch constraint:

$$\theta_4 = \varphi - \theta_2 - \theta_3$$

There is 1 solution for θ_4 per (θ_2, θ_3) pair.

Total Solutions

The total number of IK candidates is:

$$2 (\theta_1) \times 2 (\theta_3) \times 1 (\theta_2) \times 1 (\theta_4) = 4 \text{ solutions}$$

These four candidates are:

1. $\theta_1^+, \theta_3^+, \theta_2, \theta_4$
2. $\theta_1^+, \theta_3^-, \theta_2, \theta_4$
3. $\theta_1^-, \theta_3^+, \theta_2, \theta_4$
4. $\theta_1^-, \theta_3^-, \theta_2, \theta_4$

All four are passed through the filtering stage (checkJointLimits, checkSelfCollision, floor collision) and the survivor with the lowest weighted cost is selected for execution.